Master System Architecture & Connectivity Manual

A complete engineering analysis tracking file interconnections, execution pipelines, data paths, and loop logic.

---

1. Structural Map of Project Files

The system follows a strict modular structure. The files are separated into **Perfacing/Hardware Layer Interface** handlers, **Core DSP Algorithm** blocks, and **Pipeline Orchestration Layouts**.

- `audio_pipeline.c / .h` **(The Orchestrator)**: The central nerve system of the live firmware application. It manages the primary processing loops, handles global configuration variables, and links hardware components to software logic.
- `main_test.c` **(The Testing Environment)**: The desktop simulation harness. It links to standard audio libraries to read and write audio files, helping verify the math logic before deploying code to the microcontroller hardware.
- `gain.c / .h` **(DSP Linear Multiplier)**: A memoryless attenuation and boost stage. It scales audio frame amplitudes cleanly based on raw numbers.
- `equalizer.c / .h` **(DSP Parallel Filter Matrix)**: Manages the 3-band parallel configuration layout. It copies input data arrays into discrete sub-filters to process Bass, Mids, and Treble ranges independently.
- `delay.c / .h` **(DSP Circular Displacer)**: Manages the time-shifting ring buffer memory. It handles sequential read and write operations inside an isolated buffer array to delay the output stream.
- `limiter.c / .h` **(DSP Safe Amplitude Clipper)**: The final protection stage. It checks wave peaks sample-by-sample and clamps values that exceed safe thresholds to prevent clipping.

2. The Data Path: How Signals Move Through Code

Every audio block follows a fixed path through the system. Below is the step-by-step path a 256-sample block takes inside the core real-time processing routine:

Stage 1: Hardware Acquisition & Buffering

The loop starts when the microphone driver captures a block of audio from the input peripheral and writes it to `input_block`. Next, `rb_Process` passes this data to the internal hardware buffer, outputting a timed block to the array variable `rb_out`.

Stage 2: Always-On Parallel Biquad Combination

The block `rb_out` feeds into three independent biquad sub-filters simultaneously:

```
low_pass_process(&bq_lp, rb_out, lp_out, AUDIO_BLOCK_SIZE);
band_pass_process(&bq_bp, rb_out, bp_out, AUDIO_BLOCK_SIZE);
high_pass_process(&bq_hp, rb_out, hp_out, AUDIO_BLOCK_SIZE);
```

A simple combination loop normalizes and combines these parallel outputs into a balanced array baseline variable named `biquad_sum`.

Stage 3: Equalizer Processing & Channel Gains (SW1 Control)

When the equalizer state is active, the data moves through `equalizer_process`. This routine splits the signal into separate bands (`eq_low`, `eq_mid`, `eq_high`). Independent volume scales are applied to these arrays in-place before they are recombined back into the active path variable `final_out`.

Stage 4: Volume Level Adjustment (SW2 Control)

The signal moves to the gain tracking stage. If gain is enabled, `gain_process` multiplies each sample by a linear scaling factor, updating the stream array safely inside `gain_out`.

Stage 5: Pure Time-Shift Delay Application (SW3 Control)

The array moves to the time-shifting ring buffer module. When active, `delay_process` reads old data out to the speaker while updating the circular array with new microphone values. If bypassed, the routine passes `NULL` to clear out old historical values while keeping the live microphone audio stream uninterrupted.

Stage 6: Protection & Hardware Playback

Before leaving the system, the data runs through `limiter_process` to check for clipping. Finally, the safe samples are sent directly to the speaker driver via `amp_Process` to minimize playback latency.

3. File Interconnections via Function Call Contracts

The files interact through clear function call boundaries. This table lists the exact function APIs that connect the orchestrator files to the underlying algorithm code structures:

Calling File Context	Target File Executed	Exact Connecting Function Call API	Memory Interface Allocation Policy
audio_pipeline.c	equalizer.c	`equalizer_init(&eq_hdl, &eq_config);`	Passes starting filter coefficient sets down into nested tracking targets.
audio_pipeline.c	equalizer.c	`equalizer_process(&eq_hdl, biquad_sum, eq_low, ...);`	**Strictly Non-In-Place:** Requires separate output target arrays to avoid cross-channel data corruption.
audio_pipeline.c	gain.c	`gain_process(&master_gain_hdl, dsp_out, gain_out, ...);`	**In-Place Compatible:** Safely supports using the same buffer array for both input and output.
audio_pipeline.c	delay.c	`delay_process(&delay_hdl, dsp_out, delay_out, ...);`	**Strictly Non-In-Place:**

Calling File Context	Target File Executed	Exact Connecting Function Call API	Memory Interface Allocation Policy
			Modifies the index positions of its own internal circular buffer during execution.
`main_test.c`	`limiter.c`	`limiter_process(&limiter_hdl, temp, temp, block);`	**In-Place Compatible:** Reads each input sample into a local register variable before writing back to memory.

4. Core DSP Loop Logic Explanations

Below are the specific loop implementations that drive your core features. They are formatted with print-safe rules to stay well within standard page layout margins.

A. Normal Linear Gain Scaling Loop

Applies a straight linear multiplication scaling factor to each sample in the array block, avoiding complex decibel conversions during real-time tracking operations.

```
 STATUS gain_process(gain_hdl_t *phdl, const float *pfInput, float *pfOutput, uint32_t
ui32NumSamples) {
    if (phdl == NULL || pfInput == NULL || pfOutput == NULL) return STATUS_NOT_OK;

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        // Simple feed-forward multiplication loop
        pfOutput[i] = pfInput[i] * phdl->gain_linear;
    }
    return STATUS_OK;
}
```

B. Pure Time-Shift Delay Loop (Zero Echo Accumulation)

Shifts the signal by reading old data out of the buffer slot before overwriting it with the new incoming sample. This handles time displacement cleanly without feedback echo loops.

```
 STATUS delay_process(delay_hdl_t *phdl, const float *pfInput, float *pfOutput, uint32_t
ui32NumSamples) {
    if (phdl == NULL || pfOutput == NULL) return STATUS_NOT_OK;

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        float input_sample = (pfInput != NULL) ? pfInput[i] : 0.0f;

        // 1. Fetch the historic sample from the active slot
        pfOutput[i] = phdl->delay_line[phdl->write_idx];

        // 2. Overwrite the circular buffer slot with fresh data
        phdl->delay_line[phdl->write_idx] = input_sample;

        // 3. Step our pointer forward and wrap around at boundaries
        phdl->write_idx++;
```

```
        if (phdl->write_idx >= phdl->delay_samples) {
            phdl->write_idx = 0;
        }
    }
    return STATUS_OK;
}
```

C. Waveform Protection Peak Limiter Loop

Protects downstream equipment from digital clipping by clamping the alternating positive and negative peaks of the audio waveform.

```
 STATUS limiter_process(limiter_hdl_t *phdl, const float *pfInput, float *pfOutput, uint32_t
ui32NumSamples) {
    if (phdl == NULL || pfInput == NULL || pfOutput == NULL) return STATUS_NOT_OK;

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        float sample = pfInput[i];

        // Clamp the positive and negative limits of the audio waveform
        if (sample > phdl->fThreshold) {
            sample = phdl->fThreshold;     // +thresh ceiling clamp
        } else if (sample < -phdl->fThreshold) {
            sample = -phdl->fThreshold;    // -thresh floor clamp
        }

        pfOutput[i] = sample;
    }
    return STATUS_OK;
}
```

5. High-Performance Feature Set Summary

The system layout uses several key structural patterns to maximize performance on the embedded microcontroller target:

- **No Loop Fragmentation (BSS Buffer Safety)**: Audio memory tracking buffers are declared as static variables at file scope. This completely bypasses stack overflow issues and dynamic memory fragmentation risks during live operation.
- **Seamless Software Bypass Routing**: Toggling an audio feature off does not halt the main data loops. The pipeline updates internal pointer offsets smoothly in the background while passing clean audio data through instantly to maintain a continuous stream.